

Guia Técnico de Metodologia Ágil

Estruturação, planejamento, execução e governança de projetos de desenvolvimento de software

A adoção de metodologias ágeis no desenvolvimento de software exige mais do que a utilização de ferramentas de gerenciamento de tarefas. É necessário estabelecer uma estrutura clara para transformar objetivos estratégicos em entregas incrementais, rastreáveis e mensuráveis.

Este artigo apresenta os principais conceitos de organização do trabalho ágil, incluindo Initiatives, Epics, Features, User Stories, Tasks e Subtasks, além de um processo de implementação passo a passo, critérios de qualidade, práticas de planejamento, acompanhamento e melhoria contínua.

O objetivo é fornecer uma referência técnica para equipes de desenvolvimento, Product Owners, Scrum Masters, Tech Leads, arquitetos de software e gestores de tecnologia.

Palavras-chave: Agile, Scrum, Kanban, Epic, User Story, Sprint, Backlog, Gestão de Projetos, Engenharia de Software.

1. Introdução

As abordagens ágeis buscam entregar valor continuamente, responder a mudanças e reduzir os riscos associados ao desenvolvimento de produtos digitais.

Diferentemente de modelos estritamente sequenciais, o desenvolvimento ágil utiliza ciclos curtos de inspeção, adaptação e entrega incremental.

Para garantir eficiência, é importante diferenciar três dimensões:

- Estratégica: quais resultados de negócio precisam ser alcançados.
- Tática: quais funcionalidades e entregas permitirão alcançar esses resultados.
- Operacional: quais atividades técnicas precisam ser executadas.

Essa separação permite rastrear o trabalho desde uma necessidade de negócio até sua implementação.



1.1. Estrutura hierárquica recomendada

INITIATIVE

Objetivo estratégico

EPIC

Grande entrega de negócio

FEATURE

Funcionalidade do produto

USER STORY

Necessidade do usuário

TASK

Atividade de implementação

SUBTASK

Atividade técnica detalhada

Essa estrutura é uma convenção de gestão, não uma hierarquia obrigatória definida pelo Scrum. Ela pode ser adaptada conforme o tamanho da equipe, a ferramenta e a complexidade do produto.

2. Conceitos e definições

2.1. Initiative => Iniciativa

Uma Initiative representa um objetivo estratégico de negócio que pode envolver vários Epics, equipes e ciclos de desenvolvimento.

Exemplo: Modernização da plataforma de vendas digitais.

Boas práticas:

- Associar a iniciativa a um resultado mensurável.
- Definir responsáveis e indicadores de sucesso.
- Evitar iniciativas sem valor de negócio claramente identificado.
- Revisar periodicamente sua relevância estratégica.



2.2. Epic => Épico

Um Epic representa uma grande entrega funcional ou de negócio que normalmente não pode ser concluída em uma única Sprint.

Exemplo: Implementação do processo de checkout de um e-commerce.

Um Epic deve possuir objetivo, escopo, critérios gerais de conclusão, dependências relevantes e histórias associadas.

2.3. Feature => Funcionalidade

Uma Feature representa uma capacidade específica do sistema, geralmente percebida pelo usuário ou por outro consumidor do produto.

Exemplo: Pagamento por cartão de crédito.

Uma Feature pode agrupar várias User Stories. Esse nível é opcional, mas especialmente útil em produtos grandes.

2.4. User Story => História de Usuário

Uma User Story descreve uma necessidade sob a perspectiva de quem recebe o valor.

O formato mais utilizado é:

- Como [perfil do usuário],
- Quero [funcionalidade ou capacidade],
- Para [benefício ou resultado esperado].

Exemplo:

Como cliente, quero salvar meus cartões de pagamento de forma segura, para agilizar futuras compras.

Uma boa história deve seguir os princípios INVEST: independente, negociável, valiosa, estimável, pequena e testável.



2.5. Task => Tarefa

Uma Task representa uma atividade necessária para implementar uma história, corrigir um problema ou realizar outro trabalho técnico.

Exemplos:

- Desenvolver endpoint de pagamento.
- Criar componentes da interface.
- Implementar testes automatizados.
- Configurar monitoramento da integração.

As Tasks descrevem como o trabalho será realizado, enquanto as User Stories descrevem qual necessidade será atendida.

2.6. Subtask => Subtarefa

Uma Subtask representa uma decomposição operacional de uma tarefa.

Exemplo de divisão da Task "Desenvolver API de pagamento":

1. Criar contrato da API.
2. Implementar validação da requisição.
3. Integrar com o gateway.
4. Implementar tratamento de erros.
5. Criar testes unitários.

A divisão deve ajudar a equipe a executar e acompanhar o trabalho, sem gerar microgerenciamento desnecessário.

2.7. Outros tipos de trabalho

Tipo	Finalidade
Bug	Corrigir comportamento incorreto.
Spike	Investigar uma incerteza técnica ou funcional.
Enabler	Habilitar futuras entregas, como infraestrutura ou arquitetura.
Technical Debt	Tratar decisões técnicas que aumentam o custo de manutenção.
Improvement	Melhorar uma funcionalidade ou processo existente.



3. Processo de implementação passo a passo

Fluxo de trabalho recomendado

1

- *Definir objetivos estratégicos*
- *Identificar problemas de negócio, resultados esperados e indicadores.*

2

- *Criar Initiatives e Epics*
- *Organizar grandes frentes de trabalho e definir seus limites.*

3

- *Decompor em Features e User Stories*
- *Transformar grandes entregas em necessidades menores e verificáveis.*

4

- *Refinar e priorizar o Product Backlog*
- *Esclarecer requisitos, riscos, dependências e critérios de aceitação.*

5

- *Planejar a Sprint*
- *Definir Sprint Goal, selecionar itens e elaborar o plano de execução.*

6

- *Executar e acompanhar*
- *Desenvolver, testar, integrar e inspecionar o progresso diariamente.*

7

- *Validar e entregar o incremento*
- *Verificar a Definition of Done e obter feedback dos interessados.*

8

- *Inspecionar e melhorar*
- *Realizar retrospectiva, ajustar o backlog e aprimorar o processo.*



3.1. Etapa 1 => Definição estratégica

Antes de criar qualquer Epic ou User Story, a equipe deve entender o problema que pretende resolver.

Um objetivo deve conter:

- Problema ou oportunidade de negócio.
- Público impactado.
- Resultado esperado.
- Indicador de sucesso.
- Restrições e riscos relevantes.

Exemplo: Reduzir o abandono no checkout de 70% para 55% em seis meses.

3.2. Etapa 2 => Estruturação dos Epics

Os Epics devem representar conjuntos coerentes de valor.

Para cada Epic, recomenda-se documentar:

Campo	Exemplo
Identificador	EPIC-001
Título	Modernização do checkout
Objetivo	Reduzir abandono de compras
Responsável	Product Owner
Indicador	Taxa de conversão
Dependências	Gateway de pagamento
Critério de conclusão	Fluxo implantado e validado

3.3. Etapa 3 => Refinamento das User Stories

O refinamento deve garantir que as histórias estejam suficientemente claras para serem discutidas e planejadas.

Cada história deve apresentar uma necessidade de usuário, valor esperado, critérios de aceitação, dependências conhecidas e tamanho adequado para a Sprint.

3.4. Etapa 4 => Priorização do backlog

A priorização deve considerar valor de negócio, risco, urgência, dependências e esforço estimado.

Técnicas possíveis incluem:

- MoSCoW: Must, Should, Could e Won't.
- WSJF: Weighted Shortest Job First.
- Matriz de valor versus esforço.



O Product Owner é responsável pela ordenação do Product Backlog no Scrum, em colaboração com os envolvidos.

3.5. Etapa 5 => Sprint Planning

Durante o planejamento da Sprint, o Scrum Team estabelece o objetivo da Sprint, seleciona itens do Product Backlog e define um plano para entregar o incremento.

A equipe considera sua capacidade, disponibilidade, desempenho anterior e riscos conhecidos.

Não é necessário decompor todas as histórias em Subtasks antes do início da Sprint. Esse detalhamento pode evoluir durante a execução.

3.6. Etapa 6 => Execução e acompanhamento

Durante a Sprint, recomenda-se:

- Manter o trabalho visível em um quadro.
- Limitar o trabalho em progresso quando necessário.
- Integrar código frequentemente.
- Automatizar verificações de qualidade.
- Identificar impedimentos rapidamente.
- Atualizar o plano conforme novos conhecimentos surgirem.

3.7. Etapa 7 => Validação e entrega

Uma história deve atender aos critérios de aceitação relevantes, e o incremento precisa cumprir a Definition of Done.

A Sprint Review permite inspecionar os resultados com os stakeholders e adaptar o Product Backlog. Não deve ser tratada apenas como uma cerimônia de aprovação.

3.8. Etapa 8 => Melhoria contínua

Na Sprint Retrospective, a equipe avalia processos, interações, ferramentas e qualidade, identificando melhorias concretas para as próximas Sprints.



4. Exemplo completo de decomposição

INITIATIVE

INIT-001 — Transformação digital de vendas

EPIC

EPIC-001 — Checkout digital

FEATURE

FEAT-001 — Pagamento com cartão

USER STORY

US-001 — Realizar pagamento com cartão

Como cliente, quero pagar meu pedido com cartão para concluir a compra online.

TASK

TASK-001 — Implementar integração com gateway

SUB-001 — Criar contrato da API

SUB-002 — Implementar autenticação

SUB-003 — Implementar testes

4.1. Critérios de aceitação da história

Para US-001, podemos definir:

1. O cliente consegue informar os dados de pagamento por meio de um componente seguro.
2. O sistema informa se o pagamento foi aprovado ou recusado.
3. Uma transação recusada não gera pedido confirmado.
4. O sistema evita cobranças duplicadas em requisições repetidas.
5. Dados sensíveis de cartão não são armazenados indevidamente.

Esses critérios devem ser verificáveis e discutidos com o Product Owner e a equipe técnica.



5. Definition of Ready e Definition of Done

5.1. Definition of Ready (DoR)

A Definition of Ready é um acordo opcional utilizado por algumas equipes para avaliar se um item está suficientemente compreendido para ser planejado.

Não é um compromisso formal obrigatório do Scrum e não deve impedir desnecessariamente o aprendizado durante a Sprint.

Checklist de refinamento

0/5

- [ ] Objetivo e valor de negócio compreendidos
- [ ] Critérios de aceitação definidos
- [ ] Dependências relevantes identificadas
- [ ] Tamanho adequado para planejamento
- [ ] Dúvidas críticas esclarecidas

5.2. Definition of Done (DoD)

A Definition of Done estabelece um padrão compartilhado de qualidade para o incremento.

Checklist de conclusão

0/7

- [ ] Código implementado e integrado
- [ ] Code review realizado conforme política da equipe
- [ ] Testes automatizados relevantes aprovados
- [ ] Critérios de aceitação verificados
- [ ] Requisitos de segurança e desempenho aplicáveis atendidos
- [ ] Documentação necessária atualizada
- [ ] Incremento integrado e utilizável

Uma história que não atende à DoD não deve ser contabilizada como parte do incremento concluído.



6. Workflow recomendado

Um workflow deve representar o fluxo real do trabalho e permitir identificar bloqueios, gargalos e entregas concluídas.

01

Backlog

Item identificado

02

Ready

Refinado e priorizado

03

In Progress

Em desenvolvimento

04

Code Review

Revisão técnica

05

Testing

Validação e testes

06

Done

Concluído conforme DoD

Fluxo meramente ilustrativo, o número de estados deve refletir o processo real da equipe, evitando etapas redundantes.

Um item bloqueado pode receber um marcador específico, preservando seu estado atual e tornando o impedimento visível.



7. Métricas de acompanhamento

As métricas devem apoiar decisões e melhorias, e não servir como instrumento isolado de avaliação individual.

Métrica	Objetivo
Lead Time	Tempo entre a solicitação e a entrega.
Cycle Time	Tempo entre o início do trabalho e sua conclusão.
Throughput	Quantidade de itens concluídos por período.
WIP	Quantidade de itens em andamento.
Sprint Goal Success	Acompanhamento do alcance do objetivo da Sprint.
Defect Rate	Acompanhamento de defeitos.
Velocity	Apoio ao planejamento da própria equipe, quando utiliza Story Points.

7.1. Interpretação das métricas

Um aumento constante no Cycle Time pode indicar gargalos de revisão, excesso de trabalho simultâneo, dependências externas ou histórias grandes demais.

A Velocity não deve ser utilizada para comparar produtividade entre equipes, pois Story Points são estimativas relativas e locais.

8. Melhores práticas de engenharia e gestão ágil

- ✓ *Priorizar entregas de valor*
- ✓ *Organizar o backlog por resultados de negócio, e não apenas por componentes técnicos.*
- ✓ *Manter histórias pequenas*
- ✓ *Dividir funcionalidades em incrementos verificáveis, reduzindo riscos e tempo de feedback.*
- ✓ *Automatizar a qualidade*
- ✓ *Utilizar integração contínua, testes automatizados, análise estática e verificações de segurança.*
- ✓ *Reduzir trabalho em progresso*
- ✓ *Concluir atividades iniciadas antes de abrir excessivamente novas frentes.*
- ✓ *Garantir rastreabilidade*
- ✓ *Relacionar histórias, código-fonte, pull requests, testes e entregas sempre que útil.*
- ✓ *Inspecionar e adaptar*
- ✓ *Revisar regularmente prioridades, práticas de trabalho e indicadores.*



9. Erros comuns e ações corretivas

Erro	Ação recomendada
Epic grande demais e sem objetivo claro	Dividir por capacidades ou resultados de negócio.
User Story descrita como tarefa técnica	Reescrever sob a perspectiva de valor para o usuário.
Criar Subtasks excessivamente detalhadas	Detalhar apenas o necessário para coordenação.
Planejar 100% da capacidade da equipe	Considerar incertezas, suporte e interrupções previsíveis.
Usar Story Points como medida individual	Utilizar estimativas apenas para planejamento e aprendizado.
Mover itens para Done sem validação	Aplicar consistentemente a DoD.
Acumular muitos itens em andamento	Estabelecer políticas de WIP e priorizar conclusão.
Tratar a Sprint Review como aprovação burocrática	Focar no feedback e na adaptação do produto.

10. Plano de implantação em quatro semanas

Semana 1

Diagnóstico e definição

Mapear o processo atual, identificar problemas, definir papéis e acordar a hierarquia de trabalho.

Semana 2

Estruturação do backlog

Criar Epics, decompor histórias prioritárias, definir critérios de aceitação e organizar a ferramenta.

Semana 3

Execução piloto

Executar um ciclo de trabalho com acompanhamento diário, políticas explícitas e verificação de qualidade.

Semana 4

Avaliação e ajustes

Inspecionar resultados, coletar feedback, revisar métricas e ajustar o processo.

Esse roteiro é uma sugestão inicial, não uma prescrição do Scrum. A duração e o escopo devem ser ajustados à maturidade da organização.



11. Conclusão

Uma estrutura ágil eficiente conecta a estratégia do negócio à execução técnica, mantendo transparência sobre o trabalho, qualidade das entregas e capacidade de adaptação.

A hierarquia *Initiative → Epic → Feature → User Story → Task → Subtask* oferece uma forma útil de organizar produtos complexos, mas deve ser simplificada sempre que níveis adicionais não agregarem valor.

A maturidade ágil não é determinada pela quantidade de cerimônias, ferramentas ou itens cadastrados. Ela se manifesta na capacidade de entregar incrementos utilizáveis, aprender com feedback, melhorar continuamente e produzir resultados de negócio.

12. Referências técnicas

1. Schwaber, K.; Sutherland, J. The Scrum Guide. 2020. https://scrumguides.org/.
2. Beck, K. et al. Manifesto for Agile Software Development. 2001. https://agilemanifesto.org/.
3. Anderson, D. J.; Carmichael, A. Essential Kanban Condensed. 2016.
4. Cohn, M. User Stories Applied: For Agile Software Development. Addison-Wesley, 2004.
5. Forsgren, N.; Humble, J.; Kim, G. Accelerate: The Science of Lean Software and DevOps. IT Revolution, 2018.

Documento técnico de referência aplicável a equipes de desenvolvimento de software, produtos digitais e gestão ágil de projetos.

EducaCiência FastCode para a comunidade